

LAB PRACTICAL: Constructors & Inheritance

Theory Example : Program 1: Employee Constructor

Task

Run the program **as it is**.

```
class Employee {  
    int empId;  
    String name;  
  
    Employee(int id, String n) {  
        empId = id;  
        name = n;  
        System.out.println("Employee constructor executed");  
    }  
}
```

Do not add main yet.

Program 2: Manager Inherits Employee

Task

Add the following class **below Employee** and run.

```
class Manager extends Employee {  
    String department;  
  
    Manager(int id, String n, String dept) {  
        super(id, n);  
        department = dept;  
        System.out.println("Manager constructor executed");  
    }  
  
    public static void main(String[] args) {  
        Manager m = new Manager(101, "Ravi", "HR");  
  
        System.out.println("ID: " + m.empId);  
        System.out.println("Name: " + m.name);  
        System.out.println("Department: " + m.department);  
    }  
}
```

Observation Questions (Answer After Running)

1. Which constructor executed first?
2. Did you explicitly create an Employee object?
3. Why are empId and name accessible in Manager?
4. Who initializes employee details — Employee or Manager?

Program 3: Remove super() (Mandatory Experiment)

Task

Remove this line from Manager constructor:

super(id, n);

Now compile and run.

Questions (Based on Error)

1. What compilation error occurs?
 2. Why does Java not allow object creation here?
 3. Does Java automatically call parent constructor in all cases?
 4. When does super () become compulsory?
-

Program 4: Default Constructor Case

Task

Modify Employee as follows:

```
class Employee {  
    int empId;  
    String name;  
  
    Employee() {  
        System.out.println("Employee default constructor");  
    }  
}
```

Keep Manager constructor **unchanged**.

Questions

1. Does the program run now?
2. Which constructor is called implicitly?
3. Why did the program work without writing super()?
4. What difference do you observe compared to Program 3?

Program 5: Multiple Manager Objects

Task

Modify main as follows:

```
public static void main(String[] args) {  
    Manager m1 = new Manager(101, "Asha", "Finance");  
    Manager m2 = new Manager(102, "Rohit", "IT");  
}
```

Questions

1. How many times is Employee constructor executed?
 2. How many times is Manager constructor executed?
 3. Does constructor execution depend on class or object?
 4. Can constructor execution be skipped?
-

Post-Lab Questions

1. Why must Employee constructor execute before Manager?
2. Why are constructors not inherited?
3. What problem occurs if Employee has no constructor?
4. Why is super() always the first statement?
5. What real-world relationship does Employee-Manager represent?
6. Can Manager exist without Employee initialization?
7. Why are constructors executed automatically?

Different Examples

EXPERIMENT 1

Constructor Execution Order

Concept

Parent constructor always executes **before** child constructor.

Program 1

```
class Vehicle {  
    Vehicle() {  
        System.out.println("Vehicle constructor");  
    }  
}
```

```
class Car extends Vehicle {  
    Car() {  
        System.out.println("Car constructor");  
    }  
}
```

```
public static void main(String[] args) {  
    Car c = new Car();  
}
```

Observations

1. Which constructor executes first?
 2. Did you explicitly create a Vehicle object?
-

EXPERIMENT 2

Parent Dependency (Compile-Time Failure)

Concept

Child **cannot exist** without satisfying parent constructor.

Program 2A (WILL FAIL)

```
class Animal {  
    Animal(String type) {  
        System.out.println("Animal type: " + type);  
    }  
}
```

```
class Dog extends Animal {  
    Dog() {  
        System.out.println("Dog created");  
    }  
}
```

```
public static void main(String[] args) {  
    Dog d = new Dog();  
}  
}
```

Program 2B (FIXED VERSION)

```
class Animal {  
    Animal(String type) {  
        System.out.println("Animal type: " + type);  
    }  
}  
  
class Dog extends Animal {  
    Dog() {  
        super("Mammal");  
        System.out.println("Dog created");  
    }  
  
    public static void main(String[] args) {  
        Dog d = new Dog();  
    }  
}
```

Observations

1. Why does Program 2A fail?
 2. Why does Program 2B work?
 3. Who decides the animal identity?
-

EXPERIMENT 3

Default Constructor Trap (VERY IMPORTANT)

Concept

Java provides a default constructor **only if no constructor is written**.

Program 3A (WORKS)

```
class Student {  
    int id;  
}  
  
class CollegeStudent extends Student {  
    public static void main(String[] args) {  
        CollegeStudent s = new CollegeStudent();  
        System.out.println("Object created");  
    }  
}
```

Program 3B (FAILS)

```
class Student {  
    int id;  
  
    Student(int x) {  
        id = x;  
    }  
}  
  
class CollegeStudent extends Student {  
    public static void main(String[] args) {  
        CollegeStudent s = new CollegeStudent();  
    }  
}
```


Program 3C (FIXED)

```
class Student {  
    int id;  
  
    Student(int x) {  
        id = x;  
    }  
}  
  
class CollegeStudent extends Student {  
  
    CollegeStudent() {  
        super(10);  
    }  
  
    public static void main(String[] args) {  
        CollegeStudent s = new CollegeStudent();  
        System.out.println("Object created");  
    }  
}
```

Observations

1. Why does Program 3A work?
 2. What disappeared in Program 3B?
 3. Why is super(10) compulsory in Program 3C?
 4. When does Java generate a default constructor?
-

EXPERIMENT 4

(Implicit vs Explicit super())

Concept

Java inserts super() **only when possible**.

Program 4A (WORKS)

```
class Account {  
    Account() {  
        System.out.println("Account opened");  
    }  
}  
  
class SavingsAccount extends Account {  
    SavingsAccount() {  
        System.out.println("Savings account opened");  
    }  
  
    public static void main(String[] args) {  
        SavingsAccount s = new SavingsAccount();  
    }  
}
```

Program 4B (FAILS)

```
class Account {  
    Account(int balance) {  
        System.out.println("Account opened with balance " + balance);  
    }  
}  
  
class SavingsAccount extends Account {  
    SavingsAccount() {  
        System.out.println("Savings account opened");  
    }  
  
    public static void main(String[] args) {  
        SavingsAccount s = new SavingsAccount();  
    }  
}
```

Program 4C (FIXED)

```
class Account {  
    Account(int balance) {  
        System.out.println("Account opened with balance " + balance);  
    }  
}  
  
class SavingsAccount extends Account {  
    SavingsAccount() {  
        super(1000);  
        System.out.println("Savings account opened");  
    }  
  
    public static void main(String[] args) {  
        SavingsAccount s = new SavingsAccount();  
    }  
}
```

Observations

1. Why does Program 4A work without super()?
2. Why does Program 4B fail?
3. Why does Java stop inserting super() automatically?
4. When is super() compulsory?